

OWASP ZAP — Study Guide

OWASP ZAP (Zed Attack Proxy) is the leading **free, open-source web application security testing tool** — functionally the open-source counterpart to Burp Suite. It combines an intercepting proxy, automated scanner, spider/crawler, and a large plugin ecosystem into one platform, and is genuinely one of the highest-value tools to master deeply for web-focused pentesting work.

1. What ZAP Actually Is — The Core Concept

ZAP sits **between your browser and the target website** as a proxy — every request/response passes through it, letting you observe, intercept, modify, and replay traffic manually, in addition to running automated scans. This is fundamentally different from Nikto/Nuclei/Wapiti, which just fire requests at a target programmatically — ZAP is built around **you actively driving the testing**, with automation as a supporting feature, not the whole point.

Nikto/Nuclei/Wapiti → automated tools you run and read output from
OWASP ZAP → an interactive testing platform you actively work within

2. Core Components

Component	Purpose
Proxy	Intercepts/modifies traffic between browser and target
Spider	Automatically crawls a site to discover pages/links
AJAX Spider	Crawls JavaScript-heavy sites using a real browser engine (Selenium-based) — handles what Wapiti's crawler can't
Active Scan	Automated vulnerability testing against discovered pages (similar concept to Wapiti, more mature/configurable)
Passive Scan	Analyzes traffic as it flows through, without sending extra requests — flags issues just from observing normal browsing
Fuzzer	Manual payload injection into specific parameters you choose
Break points	Pause and manually edit a request/response before it's sent/received

3. Setup — Configuring Your Browser to Use ZAP as a Proxy

1. Launch ZAP (default proxy: 127.0.0.1:8080)
2. Configure your browser's proxy settings to point to 127.0.0.1:8080
3. Install ZAP's Root CA certificate in your browser (needed to intercept HTTPS traffic)
4. Browse the target site normally through the proxied browser
5. Watch ZAP's "Sites" panel populate with everything you visit

Firefox is commonly preferred for this since its proxy settings are cleanly separable from your system-wide settings — useful so your normal browsing doesn't also get routed through ZAP.

4. Automated Scan (Quick Start)

1. Launch ZAP → "Automated Scan" from the home screen
2. Enter target URL
3. ZAP spiders the site, then runs an Active Scan automatically
4. Review results in the "Alerts" tab, sorted by risk level

This is the fastest path to results and roughly comparable to what Nikto/Wapiti do — but ZAP's real value is in the manual/interactive workflow below.

5. Manual Testing Workflow (Where ZAP's Real Value Is)

1. Browse the target manually through the ZAP proxy, clicking through every feature/form/page you can find (better coverage than any spider)
2. Right-click any request in the History tab → "Attack" → various options (Active Scan just this request, Fuzz, Open in Manual Request Editor)
3. Use "Break" to pause a request mid-flight and manually edit parameters before forwarding it - test for things like IDOR by changing a user_id
4. Use the Fuzzer on a specific parameter with a custom payload list
5. Review Alerts continuously as you go, not just at the end

This manual-first approach catches things pure automation misses — business logic flaws, IDOR vulnerabilities (referencing your CIA Triad guide's IDOR example), authorization bypass — none of which an automated scanner can reliably detect without understanding the application's intended behavior.

6. The Alerts Panel — Reading Findings

Risk Level	Meaning
High	Confirmed serious vulnerability (SQLi, RCE, etc.)
Medium	Real issue, moderate impact
Low	Minor issue, often informational-adjacent
Informational	Not a vulnerability itself, but useful context

Each alert includes: description, affected URL/parameter, evidence, CWE/WASC reference IDs, and a suggested solution — similar structure to Nessus/OpenVAS output but focused specifically on application-layer issues.

7. The Fuzzer — Manual Payload Testing

1. Select a request in History, right-click → "Fuzz"
2. Highlight the parameter you want to fuzz (e.g., the value in ?id=1)
3. Add a payload list (ZAP includes built-in ones - SQLi, XSS, path traversal, etc.)
4. Run the fuzzer, review response codes/lengths/content for anomalies

This is genuinely similar in concept to what Wapiti automates, but with full manual control over exactly which parameter and which payload set — useful when you have a specific hypothesis about a vulnerability rather than wanting a broad automated sweep.

8. AJAX Spider — For Modern JavaScript Apps

Tools → AJAX Spider → enter target → select browser (Firefox/Chrome headless) → Start Scan

Since AJAX Spider uses an actual browser engine (via Selenium) to render JavaScript, it can discover content and trigger application states that a traditional crawler (like Wapiti's or ZAP's own basic Spider) would completely miss on React/Angular/Vue-heavy sites — directly solving the limitation called out in your Wapiti guide.

9. Authentication in ZAP

1. Context → New Context → define the site's scope
2. Context → Authentication → configure login method (form-based, script-based,

```
etc.)
3. Context → Users → add a test user with credentials
4. Set the Spider/Active Scan to "authenticate as" that user
```

Configuring authenticated scanning properly in ZAP takes some practice but unlocks testing everything behind a login — same importance as covered in the Wapiti guide, but ZAP's authentication handling is generally more robust and flexible for complex login flows (multi-step, CSRF-token-protected, etc.).

10. ZAP as a Command-Line/Automation Tool

ZAP isn't purely GUI — it has a full API and CLI mode, useful for CI/CD integration or scripting:

```
zap-baseline.py -t https://target.com      # quick passive-only baseline scan
zap-full-scan.py -t https://target.com     # full active scan via Docker
```

```
# ZAP's Python API - useful for integrating into your own tools
from zapv2 import ZAPv2
zap = ZAPv2(apikey='your-api-key')
zap.spider.scan('https://target.com')
zap.ascan.scan('https://target.com')
```

This scriptability is worth knowing about given your existing pattern of building Python-based security tooling (SmartScan) — ZAP's API could plausibly become a component in a future project.

11. ZAP vs. Burp Suite — Quick Comparison

	OWASP ZAP	Burp Suite
Cost	Free, fully open-source	Free (Community) / Paid (Pro)
Community	Large, OWASP-backed	Larger overall user base in industry
Automation	Strong (API, CLI, CI/CD friendly)	Pro has strong automation, Community more limited
Extensibility	Marketplace add-ons	Extensive BApp Store

	OWASP ZAP	Burp Suite
Industry perception	Widely respected, especially for learning/budget-constrained work	Often considered the "industry standard" for professional engagements

Practical reality: many professionals use ZAP for learning and budget-conscious work, then transition to Burp Suite Professional once employed somewhere that provides a license — but the underlying skills (proxying, manual testing, fuzzing, understanding HTTP deeply) transfer almost entirely between the two.

12. Kill Chain / ATT&CK Mapping

Framework	Mapping
Cyber Kill Chain	Reconnaissance → Weaponization (confirmed vulns feed exploit development)
MITRE ATT&CK	T1190 (Exploit Public-Facing Application)

Quick Reference Cheat Sheet

```
Proxy: 127.0.0.1:8080 (configure browser + install ZAP root CA for HTTPS)
Automated Scan → quick spider + active scan combo
Manual browsing + Attack → best coverage, catches logic flaws
AJAX Spider → for JS-heavy/SPA applications
Fuzzer → targeted manual payload testing
Context/Users → configure for authenticated scanning
```

```
zap-baseline.py -t https://target.com # CLI passive scan
zap-full-scan.py -t https://target.com # CLI full active scan
```